

Plumblne

It reads what your project says about itself, then goes and checks whether any of it is true.

22 of the 100 most-starred installable projects on GitHub say something about themselves that is not true. That is 14,304,526 stars between them.

22 of 100. Every one of them wrong about something.

13 README links that go nowhere
every one re-checked with a real request, not a HEAD

3 links pointing at files that are not in the repository
you click, and get nothing

3 manifests whose version is not the version on the registry
counted only where the registry points back at the repo, so a shared name is not a finding

Measured 15 September 2026 against the strongest end of GitHub, not a random sample. My first pass said 89 of 100. That was wrong in the direction that flattered the tool, so I re-checked every finding by hand. Four faults in my own checks are written up in the method. The number came down 89, then 47, then 30, then 22.

The hour before you ship.

You think you are finished. Nobody has read the README since you wrote it, CI has been green for a month, and the demo link in it was renamed in March.

Green is not the same as true. Nothing in a normal pipeline reads what the documentation claims and then goes to check. So the claims rot quietly, and the first person to find out is a user.

Green is not the same as true.

Three parts. The third one is the point.

CLAIMS

What the project says

Read out of the prose and the documents. The README, the manifest, the specification, the release checklist, the test plan.

CHECKS

What is actually true

Found out separately from the claim. A registry lookup, a real web request, a file that either exists or does not.

PROOF

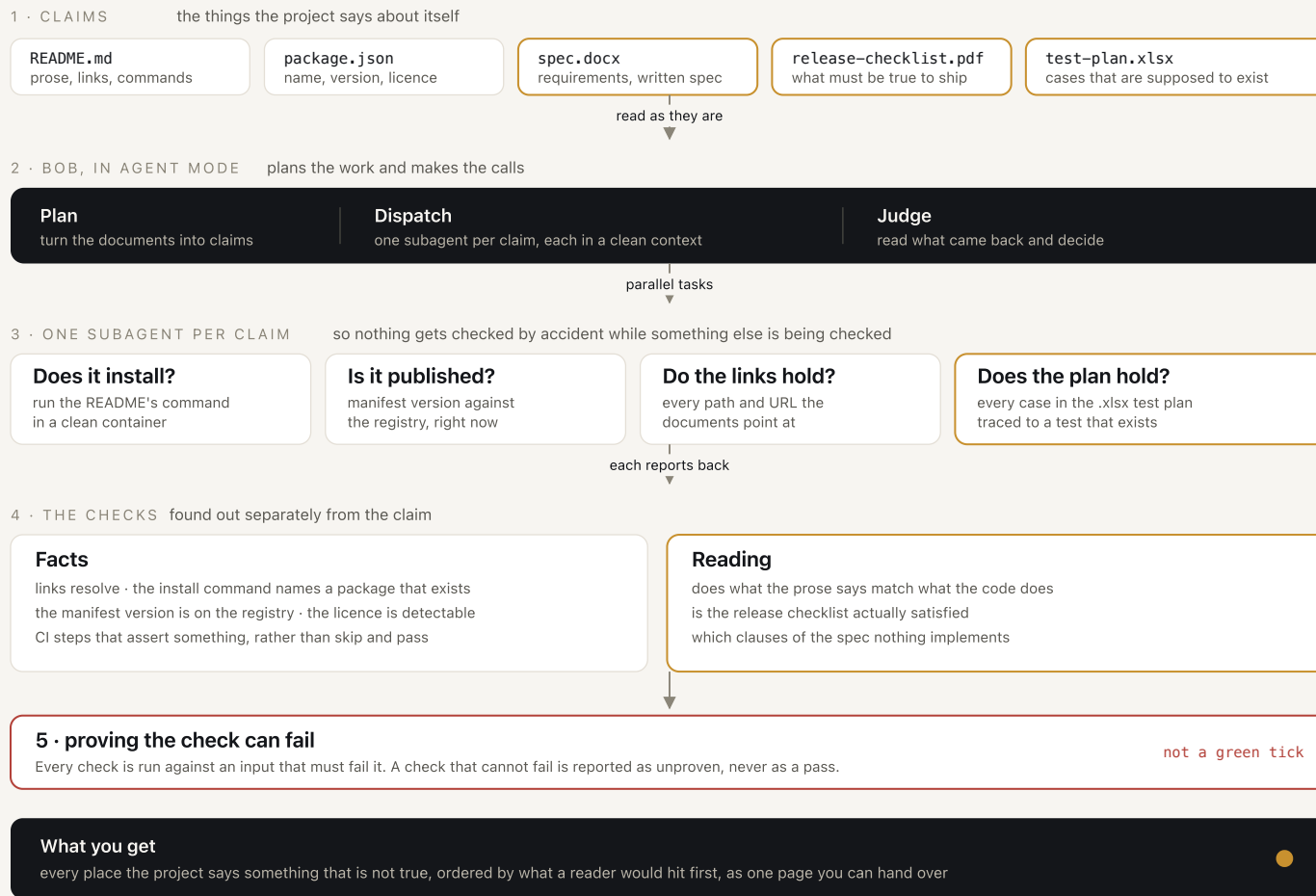
That the check can fail

A deliberate change that **must** make the check fail. If it does not, the check is reported as unproven. Never as a pass.

A check nobody has ever seen fail is not evidence. It is a green tick, and a green tick can mean nothing at all.

Plumblne

It checks whether a repository does what it says it does, and proves that each of its own checks can fail.



The half that needs no judgement has no Bob dependency and runs in CI on every push.

One README. Two filenames. One of them never existed.

```
[ADR-079] (docs/adr/ADR-079-camera-ground-truth-training.md)    exists  
[ADR-079] (docs/adr/ADR-079-camera-supervised-pose-finetune.md) never existed
```

A reader clicks that and gets a 404. The author never saw it, because they never clicked their own link after renaming the file. No test catches this. Nothing in CI reads the README.

ruvnet/RuView, checked against the GitHub contents API. Not a made-up example.

Every check proves it can fail.

Link checkers exist. Version drift tools exist. What none of them do is prove their own checks can fail before they report a pass.

I have shipped a green CI badge that was doing nothing at all. A conformance job ran nightly for days and skipped every test, because it waited on a credential that did not exist. It passed, so nothing drew attention to it. I found it on submission day.

```
$ npx github:iamrobertmoore/plumblin --selfcheck  
6/6 checks proved able to fail. Result: PROVEN.
```

Everything here that needs judgement is Bob.

DOCUMENT UNDERSTANDING

A claim can live in a .docx, a .pdf or a .xlsx, not just in markdown. A specification, a release checklist, a test plan. Bob reads those as they are.

SUBAGENTS

One subagent per claim, each with its own clean context, so nothing gets checked by accident while something else is being checked.

PARALLEL TASKS

Claims that do not depend on each other are checked at the same time. A hundred repositories do not have to queue up.

AGENT MODE

Runs the whole thing, decides what is left to check, and writes the result.

The half that needs no judgement has **no Bob dependency at all** and runs in CI on every push. If Bob is unavailable, those claims are reported as **unverified**, never as passing.

Your test plan says 84 cases. Your suite covers 61.

Bob reads the spreadsheet the way it is, traces every case to a test that exists, and names the ones nobody wrote.

No other tool can do this, because no other tool reads the plan. Every other check in Plumblin has something sitting next to it that does roughly the same job. This one has nothing.

THE CLAIM

A test plan is a document that claims a level of coverage.

THE CHECK

Trace every case to a test that exists in the suite.

WHAT YOU GET

The cases nobody wrote, named.

Release and deployment is one of the workflows they asked for.

- **The faults are invisible on your machine and obvious to anyone who opens the repo.** So users find them, not the team.
- **The half that needs no AI runs on every push,** at no cost and with no credentials.
- **The result is one self-contained page** you can hand to someone who was not in the room.
- **Nothing in a normal pipeline does this.** Linters read code. Nothing reads what the documentation claims.

MEASURED

542 s

to check 100 repositories and fetch and test **1,850 links**. Nine minutes.

About 5.4 seconds a repository, 14.2 at the worst. By hand this is a day of work, and it is the day nobody spends, which is exactly why the faults survive.

A failing check is not broken software.

A dead link in a README does not stop anyone installing the package. That is exactly why these faults survive: no test reads what the documentation claims.

The number is a snapshot, which is why it carries its date. The sample is deliberately the strongest end of GitHub, so 22 in 100 is a floor rather than an average. Workflows that can pass without checking anything are reported as warnings and kept out of the headline, because switching a docs job to `continue-on-error` is a choice someone made, not a lie.

If the tool cannot see something, it says so. It never calls it a pass and never calls it a failure. It does not guess, and it does not accuse.

Find out what you are actually shipping.

Live report: iamrobertmoore.github.io/plumblines

Source: github.com/iamrobertmoore/plumblines

Method: [docs/MEASUREMENT.md](#)

```
npx github:iamrobertmoore/plumblines --repo .
```

Two commands. Exits 0 on a pass, 1 on a failed check, 2 when a check cannot be shown to fail. No dependencies. It reads your repository and writes nothing.